

PROJECT

7.7

BUILD A CI/CD PIPELINE WITH AWS

DEVOPS SERIES

Prepared by:
Amrize JK
Certified Solutions Architect Associate

Objective

This project sets up a fully automated CI/CD pipeline using AWS developer tools like CodeArtifact, CodeBuild, and CodeDeploy. It streamlines application delivery by managing source, build, and deploy stages through CodePipeline.

The pipeline ensures reliable and repeatable deployments from development to production. It also includes testing workflows and rollback capabilities for safe and efficient software releases.

Services

Amazon EC2 - Hosts the deployed web application.

AWS CodeArtifact - Stores and manages application packages securely.

AWS CodeBuild - Compiles source code, runs tests, and produces build artifacts.

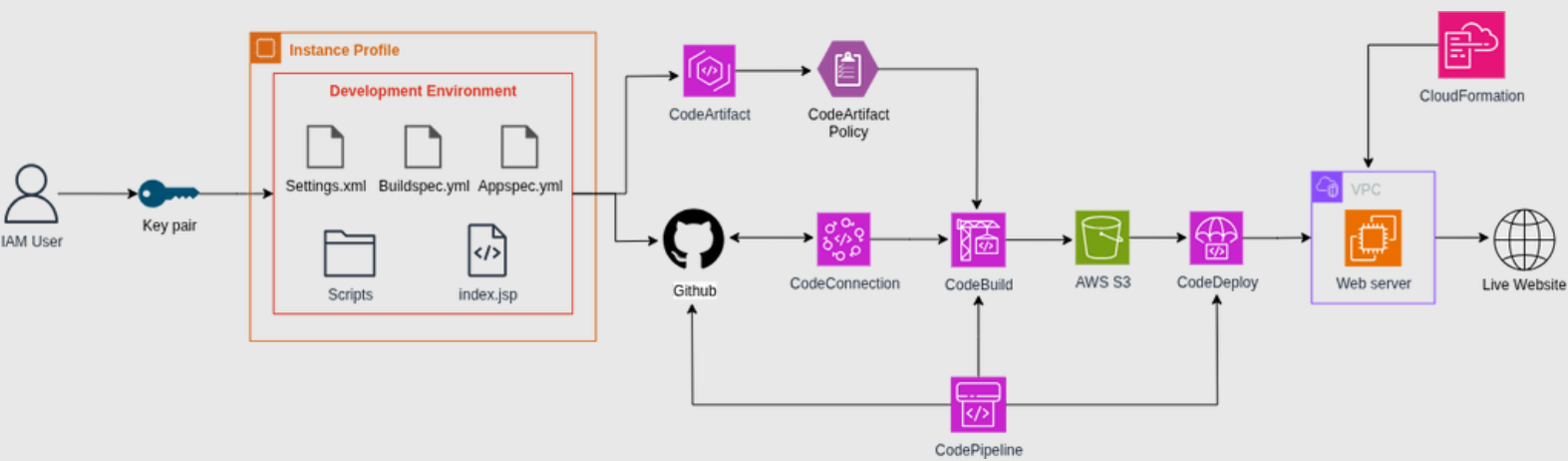
AWS CodeDeploy - Automates application deployments to Amazon EC2.

AWS CodePipeline - Orchestrates the CI/CD workflow from source to deployment.

AWS S3 - Stores pipeline artifacts and build outputs.

Amazon CloudWatch - Monitors pipeline events and logs for build and deploy.

Architecture Diagram



This architecture sets up a streamlined CI/CD pipeline using AWS services. Code from GitHub is connected via CodeConnection, dependencies are managed with CodeArtifact, and CodeBuild handles compilation. Artifacts are stored in S3 and deployed to a web server in a VPC using CodeDeploy. The entire process is orchestrated by CodePipeline and provisioned through CloudFormation, ensuring fast, reliable, and automated deployments.

Steps to Implement

1. Launch a development EC2 instance:

Create an EC2 instance with proper IAM role, tags, and security groups. This instance will act as your application host and deployment target. Ensure it's ready for SSH access and future integration.

2. Install necessary tools in EC2:

Set up Java, Apache, and other required packages inside the EC2. Place your index.jsp file and shell scripts into the server. Test everything locally to verify it works.

3. Set up IAM roles and policies:

Create IAM roles for EC2, CodeBuild, CodeDeploy, and CodePipeline. Assign minimal permissions needed for each service. This ensures secure and smooth service interaction.

4. Create a CodeArtifact repository:

Set up a domain and repository in CodeArtifact to manage dependencies. Configure it to pull packages or store your own libraries. This will be used during the build phase.

5. Upload application code to GitHub:

Push your app source code, scripts, and buildspec file to GitHub. This repo becomes the source for your CI/CD pipeline. Make sure the structure is clean and ready.

6. Connect GitHub with CodePipeline:

Use AWS CodeConnection to authorize GitHub access for CodePipeline. Select the correct repository and branch. This enables automatic triggers on new commits.

Steps to Implement

7. Create a CodeBuild project:

Define a CodeBuild project linked to GitHub as the source. Use CodeArtifact for dependencies and specify a buildspec file. Store the output artifacts in S3.

8. Set up an S3 bucket for artifacts:

Create an S3 bucket with versioning to store build outputs. This bucket acts as the bridge between CodeBuild and CodeDeploy. Grant the right access permissions to both.

9. Create a CodeDeploy application and group:

Set up CodeDeploy to target your EC2 instance using tags. Configure deployment settings like rollback and deployment type. Connect it to the S3 artifact bucket.

10. Configure the CodePipeline stages:

Add Source (GitHub), Build (CodeBuild), and Deploy (CodeDeploy) stages. Link all resources and set artifact transitions. This completes your end-to-end pipeline setup.

11. Run and validate the pipeline:

Push a new commit to GitHub to trigger the pipeline. Watch each stage in CodePipeline execute successfully. Confirm the app is deployed correctly to EC2.

12. Test rollback by failing deployment:

Make a change that causes a deployment failure to simulate an issue. CodeDeploy will trigger automatic rollback to the previous version. Verify that the app recovers smoothly.

Story Time

EC2 (The Cloud Castle for Your App):

EC2 is your cloud castle where your app lives and breathes. You connect, build, and deploy here like a digital wizard's lab.

CodeArtifact (The Potion Shelf of Dependencies):

CodeArtifact stores your magical code potions and libraries. It keeps everything ready so your spells (builds) never break.

CodeBuild (The App Chef in the Cloud):

CodeBuild is your cloud chef who follows the recipe (buildspec) perfectly. It combines your code and packages into a ready-to-serve build.

CodeDeploy (The Royal Messenger):

CodeDeploy is the fast and loyal messenger who takes your app to the castle. Whether new or rolled back, it never misses a delivery.

CodePipeline (The Spellbook of Automation):

CodePipeline is your enchanted spellbook that links every magic step. With one trigger, it commands the whole kingdom to act in sync.

GitHub (The Ancient Scroll of Code):

GitHub is the ancient scroll where your code's story is written. Every change is recorded like runes in a timeless vault.

S3 (The Treasure Chest of Artifacts):

S3 is the safe treasure chest that holds your golden builds. CodeBuild stores here, and CodeDeploy retrieves it to cast the final spell.

BY
AMRIZE J.K

THANK YOU!

